🚀 CP Bootcamp 2026 — Day 11

Theme: Stack, Queue + Data Structure Thinking

> **Main Goal:**
> আজ থেকে তুমি শুধু algorithm না, **data কীভাবে store এবং manage করতে হয়** সেটা শিখবে।

এতদিন:

```
Array
String
Sorting
Searching
Recursion
```

এগুলো দিয়ে কাজ করেছো।

এখন:

```
Data
↓
How to Organize?
↓
Which Structure?
↓
Efficient Operations
```

Estimated Time: 6–7 hours

📖 Day 11 Chapter Map

```
Day 11
│
├── Chapter 1 — Data Structure Mental Model
│
├── Chapter 2 — Stack Concept (LIFO)
│
├── Chapter 3 — Stack Implementation Using Array
│
├── Chapter 4 — Stack Applications
│
├── Chapter 5 — Queue Concept (FIFO)
│
├── Chapter 6 — Queue Implementation Using Array
│
├── Chapter 7 — Deque & Priority Queue Idea
│
├── Chapter 8 — Choosing Correct Data Structure
│
├── Chapter 9 — Common Bugs & Edge Cases
│
└── Chapter 10 — Day 11 Assignment & Practice
```

Chapter 1 — Data Structure Mental Model

Data Structure কী?

Data Structure হলো:

> Data-কে এমনভাবে organize করার পদ্ধতি যাতে operation সহজ এবং efficient হয়।

Example:

তোমার কাছে:

```
10 20 30 40
```

এগুলো store করার জন্য array ব্যবহার করতে পারো।

কিন্তু যদি দরকার হয়:

```
Last inserted item আগে বের হবে
```

তাহলে array perfect না।

এখানে:

```
Stack
```

use হবে।

Algorithm + Data Structure

CP-এর মূল formula:

```
Problem

↓

Need Operation

↓

Choose Data Structure

↓

Apply Algorithm
```

Example:

Problem:

Undo system বানাতে হবে।

যেমন:

```
Text Editor

Ctrl + Z
```

Need:

```
Last action first undo
```

Pattern:

```
LIFO
```

Data Structure:

```
Stack
```

Chapter 2 — Stack Concept (LIFO)

Stack কী?

Stack হলো:

> এমন data structure যেখানে last inserted element আগে বের হয়।

একে বলে:

LIFO

```
Last In
First Out
```

Real Life Example:

Plate Stack:

```
30  ← Top

20

10
```

তুমি আগে যেটা রাখছো:

```
10
```

তারপর:

```
20
```

```
30
```

বের করবে:

```
30
```

Stack Operations

মূল operation:

1. Push

নতুন element যোগ করা।

Example:

Before:

```
10
20
```

Push:

```
30
```

After:

```
30 ← Top

20

10
```

2. Pop

Top element remove করা।

Before:

```
30

20

10
```

Pop:

Remove:

```
30
```

After:

```
20

10
```

3. Peek / Top

Top element দেখা।

কিন্তু remove না করা।

Stack Visualization

```
 TOP

  ↓

[30]

[20]

[10]
```

Chapter 3 — Stack Implementation Using Array

C-তে built-in stack নেই।

আমরা array দিয়ে বানাবো।

Structure:

```
int stack[100];

int top = -1;
```

কেন:

```
top = -1
```

?

কারণ:

Empty stack।

কোনো index নেই।

Push Operation

Add element:

```
void push(int value)
{
    top++;

    stack[top]=value;
}
```

Example:

Start:

```
top = -1
```

Push 10:

```
top = 0
```

Stack:

```
10
```

Push 20:

```
top = 1
```

Stack:

```
20
10
```

Pop Operation

Remove top:

```
int pop()
{
    int value = stack[top];

    top--;

    return value;
}
```

Example:

Stack:

```
30

20

10
```

Pop:

Return:

```
30
```

New:

```
20

10
```

Peek Operation

```
int peek()
{
    return stack[top];
}
```

Stack Overflow

যদি stack full হয়:

```
top == size-1
```

তাহলে:

আর push করা যাবে না।

Stack Underflow

যদি:

```
top == -1
```

তাহলে:

Pop করা যাবে না।

Chapter 4 — Stack Applications

Stack CP-তে অনেক জায়গায় লাগে।

1. Parentheses Matching

Problem:

Check:

```
()
{}
[]
```

balanced কিনা।

Example:

Valid:

```
({[]})
```

Invalid:

```
([)]
```

Idea:

Opening bracket:

```
push()
```

Closing bracket:

```
pop()
```

Example:

Input:

```
( [ ] )
```

Step:

```
(
push
```

Stack:

```
(
```

```
[
push
```

Stack:

```
[
(
```

```
]
pop
```

Match।

শেষে:

Stack empty হলে:

Balanced।

2. Reverse Data

Stack naturally reverse করে।

Example:

Input:

```
ABC
```

Push:

```
A
```

```
B
```

```
C
```

Pop:

```
C
B
A
```

3. Function Call Stack

তুমি recursion-এ দেখেছো।

Actually:

```
Function call
↓
Stack
```

use করে।

Example:

```
factorial(5)
factorial(4)
factorial(3)
```

সব stack-এ যায়।

Chapter 5 — Queue Concept (FIFO)

Stack:

```
Last come
First out
```

Queue:

```
First come
First out
```

FIFO

Real life:

Bus line:

```
Person 1

Person 2

Person 3
```

যে আগে এসেছে:

সে আগে যাবে।

Queue Structure

```
Front                  Rear

  ↓                      ↓

[10][20][30][40]
```

Operations:

Enqueue

Add element.

Rear side-এ।

Dequeue

Remove element.

Front side থেকে।

Chapter 6 — Queue Implementation Using Array

Variables:

```
int queue[100];

int front = 0;

int rear = -1;
```

Enqueue

```
void enqueue(int value)
{
    rear++;

    queue[rear]=value;
}
```

Example:

Enqueue 10:

```
10
```

rear:

```
0
```

Enqueue 20:

```
10 20
```

rear:

```
1
```

Dequeue

```
int dequeue()
{
    int value=queue[front];

    front++;

    return value;
}
```

Example:

Queue:

```
10 20 30
```

Dequeue:

Return:

```
10
```

Remaining:

```
20 30
```

Problem with Simple Queue

Array:

```
[ ][ ][ ][ ]
```

Add:

```
10 20 30
```

Remove:

```
10
```

Now:

```
_ 20 30
```

Space নষ্ট।

Solution:

```
Circular Queue
```

Chapter 7 — Deque & Priority Queue Idea

Deque

Double Ended Queue।

মানে:

দুই দিক থেকেই insert/remove।

```
Front  ← [10 20 30] → Rear
```

Operations:

```
push front
```

```
push back
```

```
pop front
```

```
pop back
```

Use:

- Sliding Window Maximum
- Palindrome problems

Priority Queue

Normal queue:

```
First come first serve
```

Priority Queue:

```
Highest priority first
```

Example:

Hospital:

```
Emergency patient
```

আগে যাবে।

CP-তে:

Usually:

```
Heap
```

দিয়ে implement হয়।

(এটা Day 13/14-এ আসবে)

Chapter 8 — Choosing Correct Data Structure

Problem দেখলে:

Need Last Element First?

Example:

Undo

Function call

Bracket matching

Use:

```
Stack
```

Need First Element First?

Example:

Waiting line

BFS

Use:

```
Queue
```

Need Both Direction?

Example:

Sliding window

Use:

```
Deque
```

Need Highest Priority?

Example:

Minimum/Maximum repeatedly

Use:

```
Priority Queue
```

Chapter 9 — Common Bugs & Edge Cases

Stack Bug 1

Pop empty stack।

Wrong:

```
pop();
```

when:

```
top == -1
```

Stack Bug 2

Overflow check না করা।

Before push:

Check:

```
top < size-1
```

Queue Bug 1

Empty queue:

```
front > rear
```

Queue Bug 2

Index management ভুল।

Queue:

`front`

`rear`

দুটো maintain করতে হয়।

Queue Bug 3

Circular queue না বুঝে normal queue use করা।

Large operation-এ সমস্যা হবে।

Chapter 10 — Day 11 Assignment

Part A — Stack Implementation

নিজে implement করো:

1. `push()`
2. `pop()`
3. `peek()`
4. `isEmpty()`
5. `isFull()`

Part B — Stack Problems

Solve:

1. `Reverse a string using stack`
2. `Check balanced parentheses`
3. `Convert decimal to binary using stack`

Part C — Queue Implementation

Implement:

1. `enqueue()`
2. `dequeue()`
3. `front()`
4. `isEmpty()`

Part D — Pattern Recognition

বলবে কোন structure লাগবে:

Problem 1

Undo operation:

?

Problem 2

Printer queue:

?

Problem 3

Bracket matching:

?

Problem 4

BFS traversal:

?

Glossary Update

Add:

```
Data Structure

Stack

LIFO

Push

Pop

Peek

Overflow

Underflow

Queue

FIFO

Enqueue

Dequeue

Front

Rear

Deque

Priority Queue

Circular Queue
```

Pattern Library Update

Add:

```
## Stack Pattern

Recognition:

Need last inserted item first.


Mental Model:

Push

↓

Store

↓

Pop


Common Uses:

- Bracket matching
- Undo
- Recursion
```

Queue Pattern

Recognition:

Need first inserted item first.

Mental Model:

Enqueue

↓

Wait

↓

Dequeue

Common Uses:

- BFS
- Scheduling
- Processing order

✅ Day 11 Completion Checklist

[ ] Data Structure concept বুঝি

[ ] Stack কী বুঝি

[ ] LIFO বুঝি

[ ] Push/Pop implement করতে পারি

[ ] Stack overflow/underflow বুঝি

[ ] Bracket matching solve করতে পারি

[ ] Queue কী বুঝি

[ ] FIFO বুঝি

[ ] Enqueue/Dequeue implement করতে পারি

[ ] Deque idea বুঝি

[ ] Priority Queue idea বুঝি

[ ] Problem দেখে correct structure choose করতে পারি

🏁 Day 11 Final Mental Model

```
Problem

↓

What operation is frequent?

↓

Last First?
→ Stack

First First?
→ Queue

Both Side?
→ Deque

Priority?
→ Priority Queue

↓

Choose Structure

↓

Solve Efficiently
```

Day 11 শেষে:

```
Day 1-5
Pattern Foundation

+

Day 6
Functions

+

Day 7
Searching

+

Day 8
Sorting

+

Day 9
Optimization

+

Day 10
Recursion

+

Day 11
Data Structures
```

পরের:

Day 12 — Linked List + Hashing Foundation হবে।